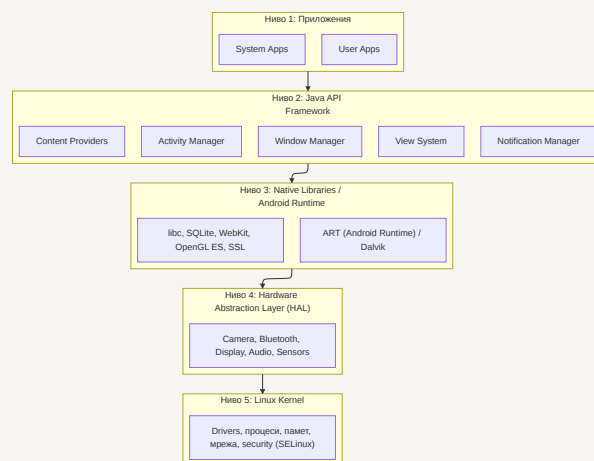


12. Архитектура и организация на ОС Android. Графичен потребителски интерфейс. Базови компоненти и техники. Диалог с потребителя.

1. Архитектура на ОС Android

Android е операционна система с отворен код, базирана на модифицирано Linux ядро, проектирана предимно за мобилни устройства. Архитектурата ѝ е **слоеста (layered)** - всеки слой предоставя услуги на слоя над него, без той да е наясно с вътрешната реализация на слоевете под себе си.



Linux Kernel - най-долното ниво. Осигурява драйвери за хардуер (камера, дисплей, Bluetooth, Wi-Fi), управление на процеси и памет, мрежов стек, power management и модел за сигурност (permissions, sandboxing на всяко приложение в отделен процес с уникален UID).

Hardware Abstraction Layer (HAL) - стандартизиран интерфейс между Java API framework и хардуерните драйвери. Позволява на framework-а да извиква

хардуерни функции, без да „знае“ конкретната им имплементация.

Native Libraries & Android Runtime (ART) - написани на C/C++: SQLite (локални БД), WebKit (browser engine), OpenGL ES (2D/3D графика), SSL. ART заменя Dalvik от Android 5.0 нататък и компилира байт-код Ahead-Of-Time (AOT) при инсталация, което подобрява производителността спрямо чисто JIT подхода на Dalvik.

Java API Framework - набор от API-та, чрез които разработчиците изграждат приложения: Activity Manager (управлява жизнения цикъл на компонентите), Window Manager, View System (за GUI), Content Providers (споделяне на данни), Notification Manager, Resource Manager.

System Apps - предварително инсталирани приложения (телефон, контакти, браузър) и потребителски приложения. И двата вида работят на едно и също ниво и използват едни и същи API.

2. Базови компоненти на Android приложения

Всяко Android приложение се изгражда от четири основни типа компоненти, деклариращи в **AndroidManifest.xml**:

Компонент	Предназначение
Activity	Един екран с потребителски интерфейс
Service	Операции на заден план (background), без да предоставя потребителски интерфейс (например: възпроизвеждане на музика или изтегляне на файл от интернет).
Broadcast Receiver	Позволява на приложението да се абонира и да реагира на системни известия или съобщения от други приложения
Content Provider	Управлява споделен набор от данни на приложението, който може да се съхранява във файловата система, SQLite база данни или мрежата. Чрез него други приложения могат сигурно да четат или записват данни.

```

<application android:label="@string/app_name">
    <activity android:name=".MainActivity">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"
/>
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <service android:name=".SyncService" />

    <receiver android:name=".BatteryReceiver">
        <intent-filter>
            <action android:name="android.intent.action.BATTERY_LOW" />
        </intent-filter>
    </receiver>

    <provider
        android:name=".NotesProvider"
        android:authorities="com.example.notesapp.provider" /
    >
</application>

```

Intent

Intent е обект-съобщение, чрез който компонентите комуникират помежду си и заявяват действие. Различаваме:

- **Явен (explicit) Intent** - сочи конкретен клас/компонент (обичайно в рамките на едно приложение)
- **Неявен (implicit) Intent** - описва действие, а системата намира подходящ компонент (в собственото или в друго приложение) чрез т.нар. *intent resolution*.

```
// Явен Intent
Intent intent = new Intent(this, DetailActivity.class);
intent.putExtra(EXTRA_NOTE_ID, noteId);
startActivity(intent);

// Неявен Intent
Intent shareIntent = new Intent(Intent.ACTION_SEND);
shareIntent.setType("text/plain");
shareIntent.putExtra(Intent.EXTRA_TEXT, "Здравей, свят!");
startActivity(Intent.createChooser(shareIntent, "Сподели чрез"));
```

3. Графичен потребителски интерфейс (GUI)

Основата на Android GUI е йерархия от обекти **View** и **ViewGroup**:

- **View** - базов градивен елемент (бутон, текстово поле, изображение).
- **ViewGroup** - контейнер (layout), който групира и позиционира други View/ViewGroup обекти - образува дърво.

Най-често използвани layout-и: **LinearLayout** (елементи в ред/колона), **RelativeLayout/ConstraintLayout** (позициониране чрез ограничения - препоръчителен подход в съвременния Android), **FrameLayout** (наслагване), **RecyclerView** (ефективен списък с преизползване на изгледи).

Обвързването на layout-а с код става чрез `setContentView()` и извличане на елементите с `findViewById()` (или чрез ViewBinding):

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

```

        Button actionButton = findViewById(R.id.actionButto
n);

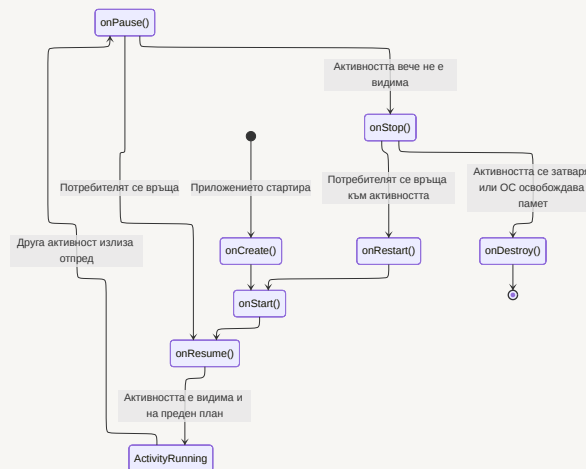
        actionButton.setOnClickListener(v ->
            Toast.makeText(this, "Натиснато!", Toast.LENGTH_S
HORT).show());
    }
}

```

4. Базови компоненти и техники - жизнен цикъл, Fragment, Adapter

Жизнен цикъл на Activity

Android управлява екраните чрез поредица от callback методи, извиквани от системата в зависимост от състоянието на Activity:



- **onCreate()** : Извиква се при първоначалното създаване на активността. Тук се извършва първоначалната настройка - дефиниране на GUI структурата, инициализиране на променливи.
- **onStart()** : Извиква се, когато активността стане видима за потребителя.

- `onResume()` : Извиква се непосредствено преди активността да започне да си взаимодейства с потребителя (тя е на преден план в стека).
- `onPause()` : Извиква се, когато друга активност застане на преден план (например диалогов прозорец). Активността е частично видима, но губи фокус. Тук се спират анимации или операции, които хаят процесорно време.
- `onStop()` : Извиква се, когато активността вече е напълно скрита от потребителя.
- `onRestart()` : Извиква се, когато активността е била спряна (`onStop()`), но потребителят се връща към нея.
- `onDestroy()` : Последното извикване преди активността да бъде унищожена от паметта.

Fragment

Fragment представлява модулна част от потребителски интерфейс, която се приема от Activity и има собствен жизнен цикъл, зависим от този на своя host (Activity). Използва се за преизползваем UI и адаптивни layout-и (напр. различно поведение на телефон и таблет), управляван от `FragmentManager`.

Adapter

Adapter е мостът между източник на данни и UI компонент за списъци (`RecyclerView` , `ListView`). `RecyclerView.Adapter` използва **ViewHolder** шаблон, за да преизползва изгледи и да намалява извиквания на `findViewById()` , което е ключово за производителността при дълги списъци.

5. Диалог с потребителя

Android предлага няколко механизма за комуникация с потребителя, различаващи се по степен на натрапчивост и изискване за реакция:

- **Toast** - кратко, ненатрапчиво съобщение, изчезва автоматично, не изисква взаимодействие.

- **Snackbar** - По-модерен еквивалент на Toast от Material Design библиотеката, който позволява добавянето на действие (Action Button), с което потребителят може да реагира (например "Undo").
- **AlertDialog** - модален диалогов прозорец със заглавие, съобщение и бутони за действие.
- **DialogFragment** - препоръчителен начин за показване на диалози, тъй като е lifecycle-aware и коректно преживява промени в конфигурацията (напр. завъртане на екрана).
- **Notification** - системно известие, показано в notification drawer, дори когато приложението не е на преден план.

```
// Toast
Toast.makeText(this, "Бележката е записана", Toast.LENGTH_SHORT).show();

// AlertDialog
new AlertDialog.Builder(this)
    .setTitle("Изтриване на бележка")
    .setMessage("Сигурни ли сте, че искате да изтриете тази бележка?")
    .setPositiveButton("Изтрий", (dialog, which) -> deleteNote())
    .setNegativeButton("Отказ", null)
    .show();
```

```
// DialogFragment - lifecycle-aware диалог
public class ConfirmDialogFragment extends DialogFragment {
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        return new AlertDialog.Builder(requireActivity())
            .setTitle("Потвърждение")
            .setMessage("Продължавате ли?")
            .setPositiveButton("Да", (d, w) -> { /* действие */ })
    }
}
```

```

        .setNegativeButton("He", null)
        .create();
    }
}

// Извикване от Activity:
new ConfirmDialogFragment().show(getSupportFragmentManager(),
"confirm");

```

```

// Notification (изисква канал от Android 8.0 / API 26 нататъ
к)
NotificationChannel channel = new NotificationChannel(
    "channel_id", "Известия", NotificationManager.IMPORTANCE_
DEFAULT);
notificationManager.createNotificationChannel(channel);

Notification notification = new NotificationCompat.Builder(th
is, "channel_id")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("Нова бележка")
    .setContentText("Бележката е записана успешно.")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build();

notificationManager.notify(NOTIFICATION_ID, notification);

```